WEEK 2 — Three Files, Three Experiments

FILE 5: `week2_ojas_recovery.py`

Job: Reproduce the main result of the paper (Figure 2)

`run_ojas_recovery(...)`

This is the full experiment — not just one trajectory but 50 trajectories, with a proper train/test split. Steps:

1. Generate 50 trajectories with known Oja's rule
2. Use 40 for training, 10 for testing (held-out)
3. Train TaylorPlasticityRule from random θ using the training set
4. After training, run the learned rule on the test set and compute R^2

R^2 (coefficient of determination) measures how well your predicted weight trajectories match the true ones. $R^2=1$ means perfect recovery, $R^2=0$ means no better than predicting the mean. The paper reports ~0.9 for clean data.

`circuit_forward_with_weights(...)`

A variant of the circuit forward pass that also returns the full weight trajectory W(t) — needed to compute R^2 against the ground-truth weights. During training you only need m(t), but for evaluation you need W(t) too.

`compute_r2(W_pred, W_true)`

Standard R^2 formula:

```
R² = 1 - SS_res / SS_tot
   = 1 - Σ(W_true - W_pred)² / Σ(W_true - mean(W_true))²
```

Computed on flattened weight matrices (all synapses, all timesteps, all test trajectories in one vector).

`plot_recovery_figure(...)`

Produces two panels that mirror Figure 2B and 2C from the paper:

- **Left panel:** Loss on log scale over epochs — should drop smoothly and plateau
- **Right panel:** All 27 Taylor coefficients over time — 25 should collapse to zero (grey lines), while θ_{110} converges to +1 (orange) and θ_{021} converges to −1 (blue)

If only one of the two Oja coefficients converges correctly, your learning rate or L1 regularization may need tuning.

`robustness_experiment()`

This is your **Figure 2D reproduction** — a 6×6 grid of R^2 values sweeping over:

- Noise levels: $\sigma \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$
- Sparsity: fraction observed $\in \{1.0, 0.9, 0.8, 0.7, 0.6, 0.5\}$

For each combination it runs a full training and records R^2. The result should show:

- High R^2 (green) in the top-left corner (low noise, full observation)
- Degrading R^2 (yellow/red) as noise increases or fewer neurons are observed
- The paper shows the model stays robust until about $\sigma=0.6$ or sparsity=0.5

This takes ~20 minutes to run. Do it overnight and save the `r2_grid` array with `np.save('r2_grid.npy', r2_grid)` so you don't have to rerun it.

FILE 6: `week2_behavioral.py`

Job: Reproduce Figure 3 — recover a reward-based rule from binary choices only

This is harder than the neural activity case because you only observe binary decisions (accept/reject), not actual neural activity.

`generate_behavioral_data(...)`

Simulates a fly doing the 2-alternative forced choice task from the paper. The structure:

- **3 blocks of 80 trials** each
- Block 1: Odor A rewarded 20% of the time, Odor B rewarded 80%
- Block 2: Reversed — A=90%, B=10%
- Block 3: Same as Block 1 again

At each trial:

1. Network computes activation for both odors with current weights
2. Softmax converts activations to choice probabilities
3. Fly "chooses" stochastically based on those probabilities
4. Reward arrives based on which odor was chosen and current block probabilities
5. Reward prediction error: `r = R - E[R]` where E[R] is a moving average of recent rewards
6. Weights update via ground-truth rule: `Δw = x_j · r`

Over time, the fly should learn to prefer the high-reward odor in each block. The choice ratio should shift from ~50% early in each block to ~80%+ toward the end.

`BehavioralCircuit`

A 3-layer circuit (2→10→1) that mirrors the mushroom body architecture. The output is a scalar probability of "choosing odor A." At each trial:

1. Input = the chosen odor's encoding (2D vector)
2. Hidden layer: `h = sigmoid(W_plastic @ x)`
3. Output: `m = mean(h)` — scalar choice probability
4. Weight update via learned rule

Crucially, training only sees the binary choices — not the underlying weights or neural activity. The rule has to be inferred from the behavioral signal alone.

`percent_deviance_explained(...)`

From paper Equation 11. Compares your model to a null model that always predicts 50% chance of choosing either odor:

```
%DevExp = 100 × (1 - Deviance_model / Deviance_null)
```

Higher is better. 0% means your model is no better than random guessing. The paper reports ~60% for the Taylor rule and ~65% for MLP on simulated data.

`run_behavioral_recovery(...)`

Runs the full behavioral recovery experiment over multiple random seeds (different weight initializations and reward sequences). Reports mean ± std of percent deviance explained. Running over 10 seeds gives you the boxplots in Figure 3F.

FILE 7: `week2_diffusion.py`

Job: The novel extension — use a diffusion model to generate synthetic trajectories, then test if inference still works

This is **your team's original contribution** beyond reproducing the paper. The question: if you train a diffusion model on Oja trajectories and sample new ones, do the generated trajectories look realistic? And can you still recover Oja's rule from them?

`NoisePredictor`

A small MLP that predicts the noise added at diffusion step t. Input is the noisy trajectory (flattened to one vector) concatenated with a timestep embedding. The embedding converts an integer t into a 32-dimensional vector — this tells the network how much noise to expect.

Architecture: (traj_dim + 32) → 256 → 256 → traj_dim

`DDPM`

Implements the forward (noising) and reverse (denoising) diffusion process:

- **Forward process** `q_sample(x0, t)`: takes a clean trajectory and adds Gaussian noise according to a schedule. At t=0 it's clean; at t=T it's pure noise.

```
x_t = sqrt(ᾱ_t) · x0 + sqrt(1-ᾱ_t) · ε,  ε ~ N(0,I)
```

- **Reverse process** `p_sample_loop()`: starts from pure noise and iteratively denoises using the trained predictor — at each step, predict and subtract the noise.

The noise schedule: β goes from 0.0001 to 0.02 linearly over 200 steps. α = 1-β. ᾱ = cumulative product of α values.

`train_ddpm(...)`

Training loop: for each mini-batch of trajectories, sample a random timestep t, add the corresponding amount of noise, run the noise predictor, compute MSE between predicted and actual noise. After 500 epochs the model learns to denoise trajectories of Oja neural activity.

`run_diffusion_experiment()`

The full experiment:

1. Generate 50 clean Oja trajectories
2. Flatten each from (T=50, n_obs=50) → (2500,) for the DDPM
3. Train DDPM on these 2500-dimensional vectors
4. Sample 20 new trajectories from the DDPM (pure noise → denoised)
5. Reshape back to (T=50, n_obs=?) and clip to [0,1]
6. Plot real vs generated side by side — do they look similar?

The **Figure 5** visualization shows 3 real trajectories on top and 3 DDPM-generated ones on bottom. If training worked, the generated ones should show similar statistical structure (activity roughly in [0.3, 0.7], smooth-ish over time) even though they're completely synthetic.

The **final comparison** (which connects to Week 3 ablations): run `run_ojas_recovery()` once with clean data and once replacing `O` with `O_synthetic` from the DDPM. Compare R^2 scores. This answers: "does inference still work when data comes from a generative model rather than the real simulator?"

How all 7 files connect

```
data_generation.py  ──────────────────────────────────────────────────────┐
  generates X, O, W_gt                                                     │
                                                                           ▼
plasticity_rules.py ──→  TaylorRule / MLPRule ──→  circuit_model.py ──→  training.py
  defines g_θ(x,y,w,r)      (27 params or MLP)     runs T-step loop       optimizes θ
                                                    keeps grad graph        via BPTT
                                                          │
                                                          ▼
                                week2_ojas_recovery.py  (Figure 2: R² + coefficient converge
                                week2_behavioral.py     (Figure 3: binary choice recovery)
                                week2_diffusion.py      (Figure 5: generative validation lay
```